Project Title:

Assistive System for Paralyzed Patients Using Motion & Wireless Communication

BME 3200: Biomedical Systems Design Project

A project submitted to the Department of Biomedical Engineering in Third Year Second Semester

Supervised by

Dr. Md. Bashir Uddin

Professor

Department of Biomedical Engineering

Submitted by

Kazi Nimat Hasan

Roll: 2015007

Department of Biomedical Engineering

Department of Biomedical Engineering

Khulna University of Engineering & Technology

Khulna-9203, Bangladesh

August 2025

ABSTRACT

This project presents a low-cost assistive communication system for patients with partial paralysis or motor impairments. The system combines an MPU6050 accelerometer for hand motion detection and a homemade flex sensor for finger flexion recognition. These signals are processed by an Arduino Uno to generate patient-specific commands such as requests for water, food, or emergency assistance. The outputs are displayed on an LCD for caregivers and transmitted wirelessly via RF and Bluetooth modules for remote monitoring. A buzzer provides audio alerts during emergencies, including fall detection. By integrating gesture-based input with wireless communication, the system enables paralyzed patients to convey essential needs more effectively, offering a scalable and affordable solution for home and hospital environments.

TABLE OF CONTENTS

	Page no
Title	1
Abstract	2
Table of Contents	3
Index	4
List of Tables	6
List of Figures	7

INDEX

CHAPTER 1	Introduction	Page
	1.1 Background	8
	1.2 Objectives	9
CHAPTER 2	Motivation/ Background Study	
	2.1: Motivation behind this project	10
	2.2: Background and Technological Context	10
	2.3: Significance of the Project	11
CHAPTER 3	Methodology	
	3.1: System design and description	13
	3.2: Module Description and Operational Flow	14
	3.3: Theoretical Background	15
CHAPTER 4	Design procedure/Experimental set-up/ Circuit Diagram	
	4.1 Materials	16
	4.2 Material Description	17
	4.3 Circuit Diagram & Connections	20
CHAPTER 5	Implementation (Step by Steps with a pictorial view)	
	5.1 Implementation	22
	5.2 Outcome Analysis	22
CHAPTER 6	Results and Discussions	
	6.1 Results	26
	6.2: Threshold value calculation	26
	6.3 Discussion	28

CHAPTER 7	**Conclusion and Future Work**	
	7.1 Conclusion	30
	7.2 Future Work	30
	References	32
	Appendix	33

LIST OF TABLES

Table no	Description	Page
4.1	Material selection and justifications	16
6.1	Final output results of my project	26

LIST OF FIGURES

Figure No	Description	Page
3.1	Block diagram of methodology of automated paralysis patient monitoring.	13
4.1	Breadboard setup	17
4.2	MPU6050 Accelerometer	17
4.3	RF Transmitter and Receiver Module	18
4.4	Arduino UNO	18
4.5	LCD Display	18
4.6	Flex Sensor	19
4.7	Bluetooth Module HC-05	19
4.8	Circuit diagram of the system	20
5.1	Implemented Device Showing (a) Transmitter Module and (b) Receiver Module	22
5.2	Output Messages Displayed on LCD for Basic Patient Needs	23
5.3	Emergency and Fall Detection Outputs with Buzzer Alerts	24
5.4	Bluetooth Terminal Receiving Real-Time Patient Commands	25

Chapter 1

Introduction

1.1 Background

Paralysis is the loss of voluntary muscle function due to disruption of the nervous system [1]. In a healthy individual, motor commands travel from the brain through peripheral nerves to initiate movement. However, in cases of paralysis, these neural pathways are damaged or blocked, resulting in partial or complete loss of mobility[2]. Common signs of paralysis include spam, loss of feeling in the arms and legs, a decline in muscular and motor function, and difficulty in speaking. While certain types of paralysis can result in serious problems, others might allow the patient to live a normal life[3]. Depending on the severity and location, paralysis can be classified as monoplegia, hemiplegia, paraplegia, diplegia, or quadriplegia [4].

- *Diplegia:* Paralysis on the same region of both sides of the body, such as both arms or legs.
- *Hemiplegia*: Paralysis on one side of the body, typically an arm and a leg.
- *Monoplegia*: Paralysis confined to one limb (arm or leg).
- *Paraplegia*: Paralysis affects both legs and, in some cases, the lower torso.
- *Quadriplegia* (*Tetraplegia*): Complete paralysis of all four limbs, often including little or no movement below the neck.

While patients with complete immobility (e.g., quadriplegia) face extensive challenges, individuals with partial or localized paralysis often retain some motor activity in their hands or other body parts. For such patients, enabling communication through residual movements offers a practical and low-cost assistive solution. This project focuses on creating a system that uses hand gestures and finger flexion as a medium of communication for patients with partial paralysis.

The system is intended primarily for home-based care, where continuous caregiver attention is not always possible, but it can also be extended to hospital environments. By integrating simple sensors with wireless communication, the project addresses a critical gap: allowing patients with limited mobility to express their essential needs independently.

1.2 Objectives

The objectives of this project are mainly:

- Developing a low-cost assistive communication system that enables paralysis patients to express their essential needs using hand gestures and finger flexion.
- Ensuring patient safety and timely assistance through real-time monitoring, fall detection, and emergency alert mechanisms.
- Enhancing quality of life and reduce caregiver burden by providing an intuitive, reliable, and scalable system for patient–caregiver interaction

This project provides a practical and cost-effective solution to enhance communication for patients with partial paralysis, bridging the gap between patient needs and caregiver response while promoting independence, safety, and improved quality of life.

Chapter 2

Motivation and Background study

2.1 Motivation Behind the Project

Communication is a fundamental human need, yet individuals with partial paralysis often face significant challenges in expressing their basic requirements due to impaired motor functions or speech limitations. Patients with conditions such as monoplegia or hemiplegia may retain partial limb mobility, but conventional communication methods remain inaccessible or insufficient. This limitation can lead to frustration, reduced autonomy, and delayed response to essential needs, which can further impact health and quality of life [5].

The motivation for this project arises from the necessity to bridge the communication gap for such patients, enabling them to convey essential commands using simple, intuitive hand gestures and finger movements. By leveraging residual motor capabilities, the system provides a practical solution that enhances independence, dignity, and timely assistance. Additionally, the use of low-cost, readily available components ensures accessibility and scalability, making it suitable for both home and hospital environments [6].

Beyond individual patient support, the project also addresses caregiver burden. Caregivers often manage multiple responsibilities and may not always be immediately available to monitor patients continuously. Integrating a real-time alert system with wireless communication ensures that caregivers are promptly notified of urgent needs or emergencies, thus reducing response time and improving patient safety[7].

2.2 Background and Technological Context

Recent advancements in microcontrollers, sensors, and wireless communication have made it feasible to develop compact, cost-effective assistive devices. In particular, the MPU6050 accelerometer and gyroscope sensor offers precise detection of hand movement and orientation, enabling gesture-based command recognition. Flex sensors provide an additional input modality, detecting finger flexion to signify emergency commands. These sensors, when combined with a

microcontroller such as the Arduino Uno, form the basis of an intelligent interface between the patient and the caregiver.

Wireless communication modules, including RF transmitters/receivers and Bluetooth (HC-05), allow the system to transmit commands to a receiver unit or smartphone in real time. This facilitates remote monitoring, logging of patient needs, and timely intervention. LCD displays at the receiver provide an intuitive interface for caregivers, while a buzzer provides immediate audio feedback for critical events such as fall detection or emergency requests [6].

Previous studies in assistive technology highlight the importance of context-aware systems that not only detect motion but also interpret user intent reliably. Systems relying solely on accelerometers may misinterpret rapid hand movements as emergency events, highlighting the need for integrating multiple sensor modalities, such as flex detection, to improve accuracy and reduce false positives. The current project adopts this hybrid approach to create a reliable and practical solution for partial paralysis patients.

2.3 Significance of the Project

The significance of this project lies in its ability to provide a low-cost, user-friendly, and scalable assistive system. The key contributions include:

- *Enhanced Patient Independence*: Patients can communicate essential needs such as requesting water, food, or washroom assistance without relying solely on caregivers.
- *Timely Emergency Alerts*: Falls or sudden distress are promptly identified using sensor thresholds, with alerts transmitted immediately to caregivers.
- *Remote Monitoring Capability*: Real-time transmission via Bluetooth allows for patient monitoring even when caregivers are not physically present, which is particularly useful in home or hospital setups.
- *Cost-Effective Implementation*: Using a custom-built flex sensor and affordable microcontrollers reduces overall cost, making the system accessible to a wider demographic.

- *Scalability and Adaptability*: The system can be further extended to incorporate additional gestures, IoT modules, or context-aware algorithms, paving the way for future enhancements.

In essence, this project not only aims to provide a practical communication tool for partially paralyzed patients but also demonstrates how low-cost electronics can be harnessed to significantly improve quality of life. By integrating motion detection, gesture recognition, and wireless communication, the system serves as a reliable, real-time bridge between patients and caregivers, promoting independence, safety, and timely assistance.

Chapter 3

Methodology

3.1 System Design and Overview

The system is designed to assist patients with partial paralysis by enabling them to communicate essential needs through hand movements and finger gestures. It consists of two main units: a Transmitter Unit and a Receiver Unit, connected wirelessly via an RF module. The total system operation is represented in Block Diagram 3.1, showing the flow from motion detection to command transmission and caregiver notification.

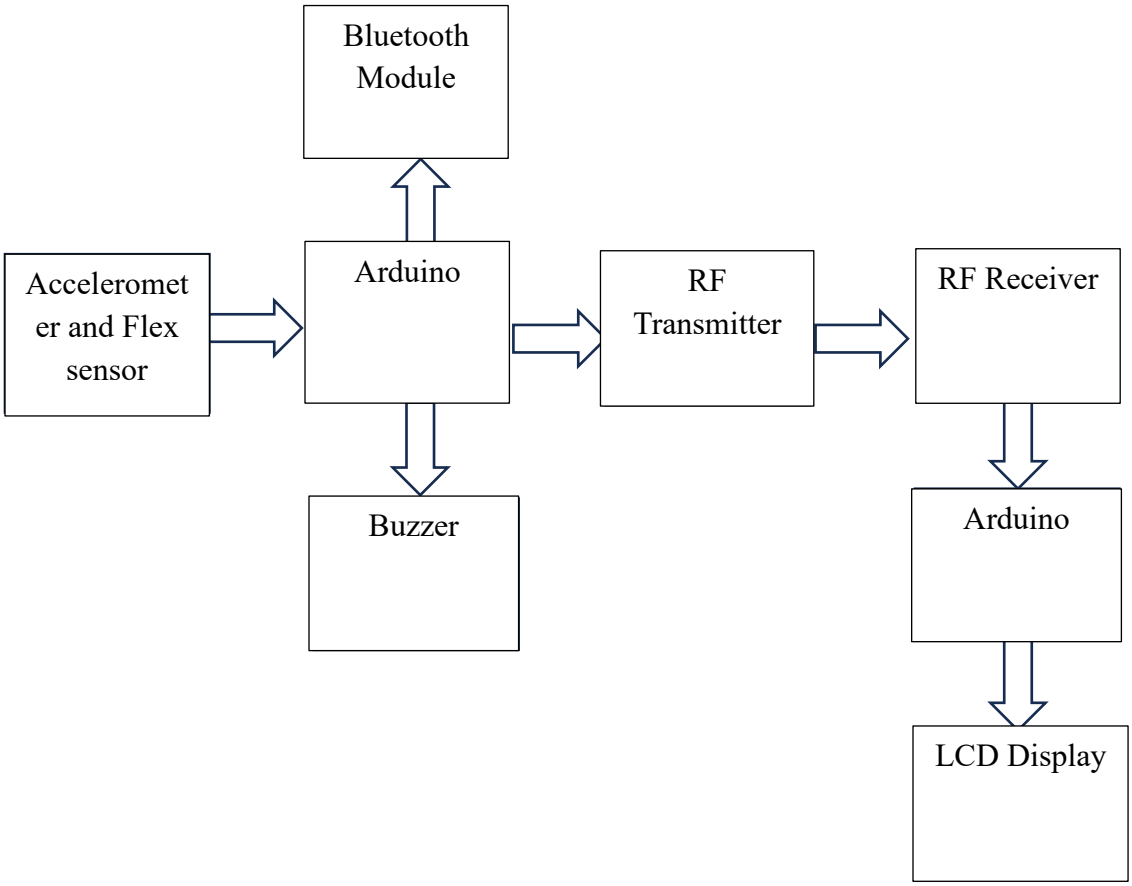


Figure 3.1 Block diagram of methodology of automated paralysis patient monitoring

3.2 Module Description and Operational Flow

Transmitter Unit:

The transmitter captures hand movements using an MPU6050 accelerometer and finger flexion via a custom-made flex sensor. The Arduino Uno processes these readings to map specific motions and gestures to predefined commands: tilting forward, backward, left, or right corresponds to "Washroom," "Need Food," "Need Water," and "Feeling Sick," respectively, while finger flexion triggers "Emergency." Sudden acceleration spikes are interpreted as a fall. The processed commands are transmitted wirelessly via the RF module and simultaneously sent to a Bluetooth-enabled terminal for remote monitoring.

Receiver Unit:

The receiver receives the commands and displays them on a 16x2 I2C LCD screen. Critical alerts such as "Emergency" or "Fall Detected" trigger a buzzer for immediate caregiver attention. Non-critical commands are recorded in the Bluetooth terminal, allowing caregivers to review patient requests even if they are momentarily inattentive.

Operational Flow:

- System initialization: Arduino calibrates the MPU6050 and initializes all components.
- Continuous monitoring: MPU6050 detects tilt and flex sensor detects finger movement.
- Command mapping: Movements are converted into predefined requests.
- Wireless transmission: Commands are sent to the receiver via RF and optionally logged on Bluetooth.
- Caregiver notification: LCD displays the command; buzzer alerts for critical commands.

3.3 Theoretical Background

The system leverages two core principles: motion sensing using an accelerometer and resistive flex sensing. The MPU6050 accelerometer measures acceleration along three axes (X, Y, Z) and angular velocity, enabling detection of hand tilt direction and movement intensity. The magnitude of acceleration is calculated as:

$$Magnitude = \sqrt{ax^2 + ay^2 + az^2}$$

Thresholds on axis differences and overall magnitude are used to detect sudden movements (fall detection) or orientation changes corresponding to patient commands.

The flex sensor operates on the principle of resistive change under bending. When a finger bends, the resistance of the sensor changes, which is read as an analog voltage by the Arduino. Specific thresholds are set to detect intentional flex gestures for emergency commands. This combination of accelerometer and flex sensor data allows reliable mapping of hand gestures to patient requests.

Wireless communication is achieved using RF modules, which provide real-time transmission of command strings from the transmitter to the receiver. The Bluetooth module adds remote monitoring capability, allowing commands to be logged on a smartphone terminal application.

Chapter 4

Materials and Connections

4.1 Materials

The selection of materials for each component of the system was carried out based on functionality, cost-effectiveness, and availability. The detailed list of materials along with their justification for inclusion in the project is summarized in Table 4.1.

Table 4.1: Material Selection and Justification

S/N	Material	Type	Justification
1	MPU6050 Accelerometer	Hardware	Detects tilt and motion in multiple axes, essential for gesture recognition.
2	Flex Sensor	Hardware	Captures finger flexion for emergency commands, enabling critical alert detection.
3	RF Transmitter-Receiver Module	Hardware	Enables wireless communication between the transmitter and receiver units.
4	Arduino Uno	Hardware	Acts as the primary microcontroller for processing and managing components.
5	I2C LCD Display (16x2)	Hardware	Displays the received commands clearly for caregivers.
6	Arduino Adapter	Hardware	Provides a stable and consistent power supply for the system
7	Bluetooth Module HC-05	Hardware	Enables real-time monitoring via a smartphone app for enhanced caregiver response.
8	Serial Bluetooth Terminal	Software	Displays real-time data sent from the HC-05 Bluetooth module for caregiver monitoring.

4.2 Material Description

A. Breadboard: Breadboards were used in the initial stages to create temporary circuits for testing and prototyping. They allow easy connection of components without soldering, making modifications quick and efficient. The power and ground strips facilitated organized wiring for testing the sensor, RF modules, and LCD display.

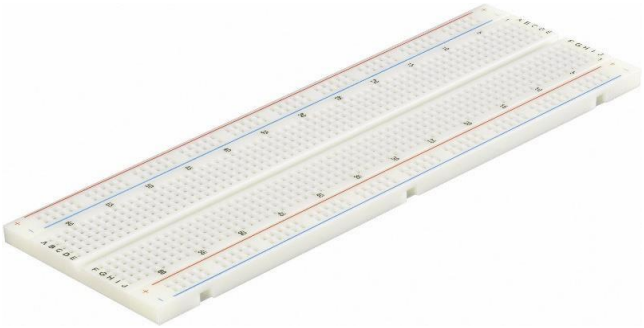


Figure 4.1 Breadboard

B. MPU6050 Accelerometer: The MPU6050 is a six-axis motion tracking device capable of detecting acceleration and angular velocity. It provides analog outputs for acceleration (X, Y, Z axes) and gyroscopic data, which are crucial for recognizing specific hand movements. The I2C interface ensures efficient communication with the Arduino Uno.

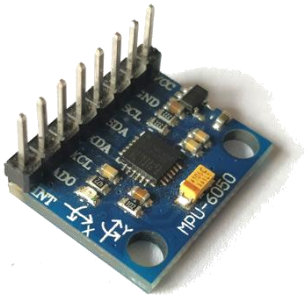


Figure 4.2 MPU6050 Accelerometer

C. RF Transmitter and Receiver Modules: The RF transmitter sends commands wirelessly to the RF receiver, enabling the two units to communicate seamlessly. These modules operate at a frequency of 433 MHz and offer a reliable range for home and hospital settings.

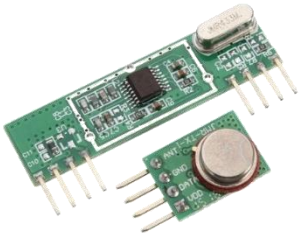


Figure 4.3 RF Transmitter and Receiver Module

D. Arduino Uno: Two Arduino Uno boards are used: one in the transmitter unit and one in the receiver unit. These microcontrollers process sensor data, map hand gestures to specific commands and manage RF communication. Their simplicity and versatility make them ideal for this application.

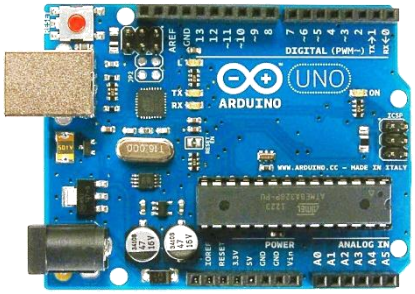


Figure 4.4 Arduino UNO

E. I2C LCD Display (16x2): The 16x2 LCD display is used to display the received commands in a clear, readable format. Its I2C interface reduces the number of pins required, simplifying the wiring process. The backlight ensures visibility in low-light environments.



Figure 4.5 LCD Display

G. Flex Sensor: The flex sensor detects finger flexion by changing its resistance when bent. It connects to the Arduino Uno via an analog pin, triggering an emergency command ("Emergency") when flexion exceeds a threshold, providing a simple method for critical alerts. Although in the project the flex sensor that was used was a home made one rather than the commercial expensive one

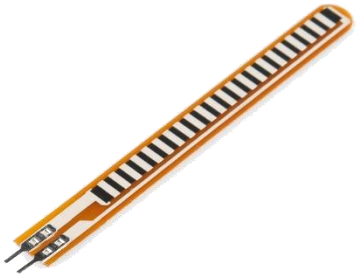


Figure 4.6 Flex Sensor

H. Bluetooth Module HC-05: The HC-05 module enables wireless communication between the Arduino and a smartphone. It transmits real-time commands to the Serial Bluetooth Terminal App, allowing caregivers to monitor patient needs remotely.

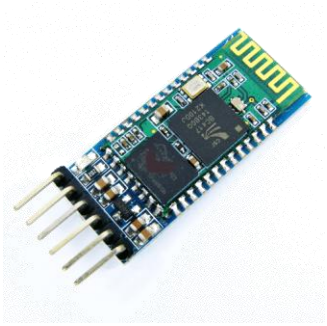


Figure 4.7 Bluetooth Module HC-05

4.3 Circuit Diagram & Connections

The complete circuit design for the implemented system has been prepared to illustrate the connections between the sensors, microcontrollers, RF modules, and output devices. The detailed circuit diagram is provided in Figure 4.9

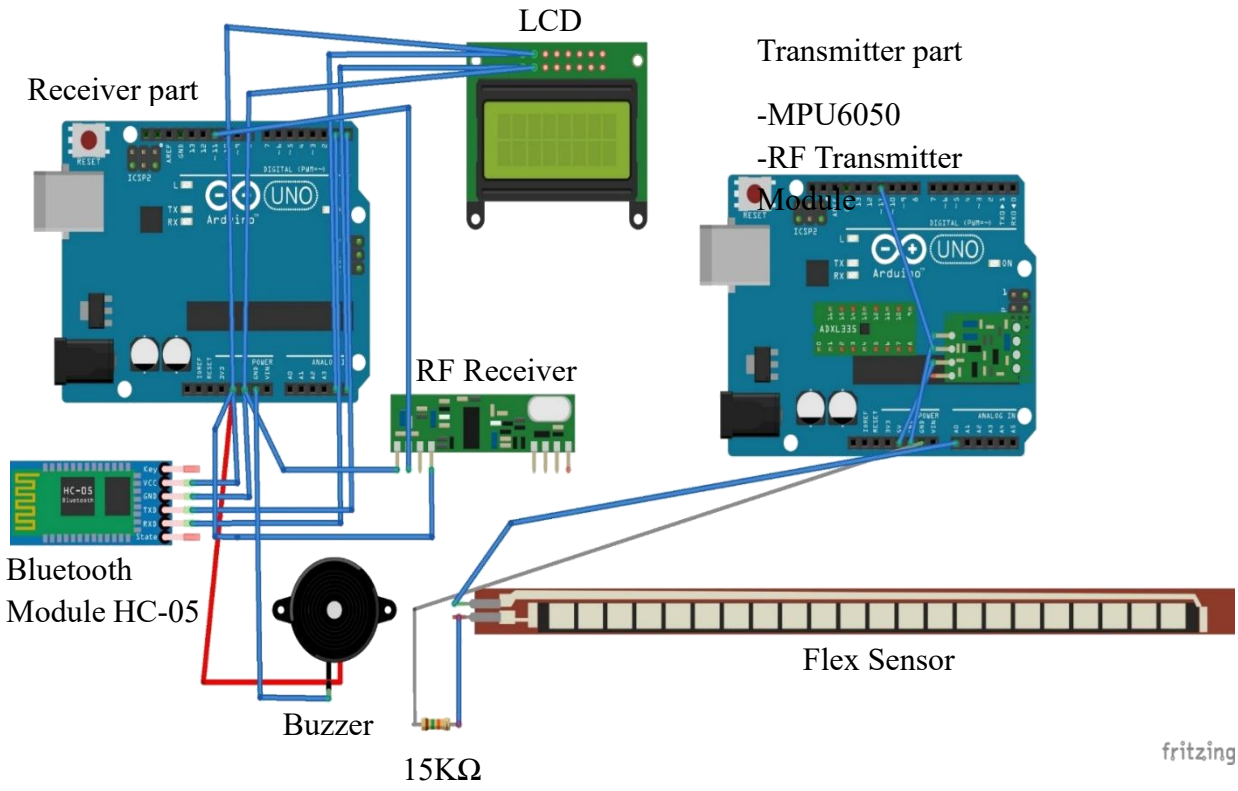


Figure 4.8: Circuit diagram of the system

The system was implemented in a structured manner, starting with the collection and testing of all components, including the MPU6050 accelerometer, flex sensor, RF modules, Arduino boards, LCD display, Bluetooth module (HC-05), and buzzer.

The Transmitter Unit was assembled by connecting the MPU6050 (SDA → A4, SCL → A5, VCC → 5V, GND → GND) and the flex sensor (one terminal → A0, other → GND via 10kΩ pull-down resistor) to the Arduino. The RF transmitter (DATA → D12) and Bluetooth module (TX → RX, RX → TX) were integrated to enable wireless communication.

The Receiver Unit comprised an RF receiver (DATA → D11), I2C LCD (SDA → A4, SCL → A5), and buzzer (D8 → positive, GND → negative), connected to a separate Arduino board.

Software development involved writing separate Arduino codes for the transmitter and receiver. The transmitter processes accelerometer and flex sensor inputs to generate commands and transmits them via RF and Bluetooth. The receiver interprets these commands, displays them on the LCD, and triggers the buzzer for emergency alerts.

Finally, the system underwent testing and calibration. MPU6050 thresholds were calibrated for accurate gesture detection and fall detection. The flex sensor was tested for reliable finger flexion detection. RF and Bluetooth communications were validated for seamless transmission and real-time monitoring. Integration of both units ensured accurate command display, buzzer alerts for critical events, and consistent system functionality.

Chapter 5

Implementation and Output

5.1 Implementation

The system was implemented successfully in hardware and divided into two main functional units: the transmitter module and the receiver module. Both parts together form the complete assistive communication device, as shown in Figure 5.1(a) and Figure 5.1(b).

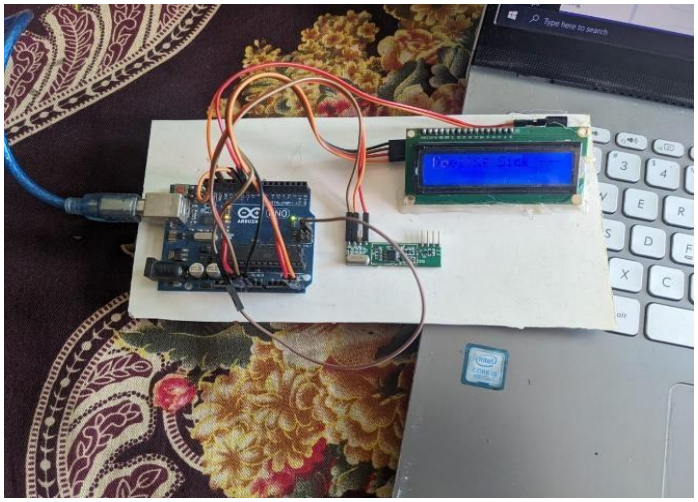


Figure 5.1(a): Receiver Module



Figure 5.1(b): Transmitter Module

Figure 5.1: Implemented Device Showing (a) Transmitter Module and (b) Receiver Module

5.2 Outcome Analysis

For outcome analysis, the system was tested under different simulated patient conditions. When specific hand gestures or finger flexions were performed, the corresponding patient needs—such as “No need,” “Feeling sick,” “Need water,” “Need food,” and “Washroom”—were accurately detected and displayed, as shown in Figure 5.2.



Fig 5.2(a) In Central Position



Fig 5.2(b) Left Position



Fig 5.2(c) Forward Position

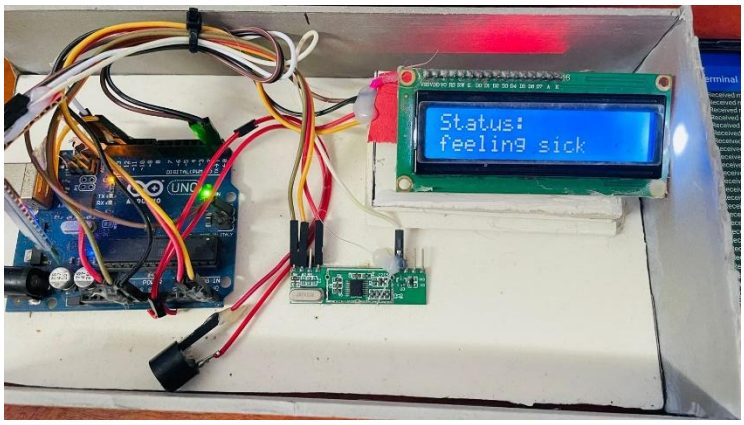


Fig 5.2(d) Hand in Right Position



Fig 5.2(e) Forward Position

Figure 5.2: Output Messages Displayed on LCD for Basic Patient Needs

Emergency conditions were also verified. In cases of sudden hand movements or critical finger flexion indicating distress, the system generated commands such as “Emergency” and “Fall Detected.” These outputs were successfully displayed on the LCD while triggering the buzzer for immediate caregiver attention, as illustrated in Figure 5.3.

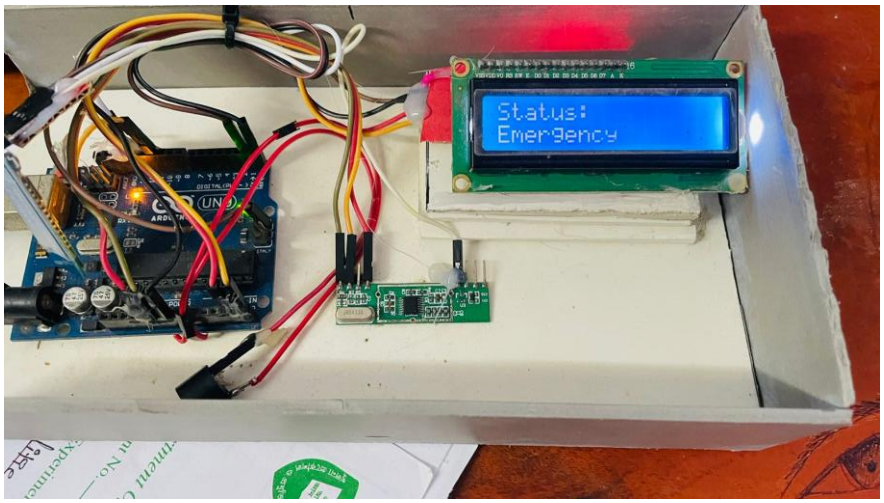


Figure 5.3(a): Emergency Output with Buzzer Alert

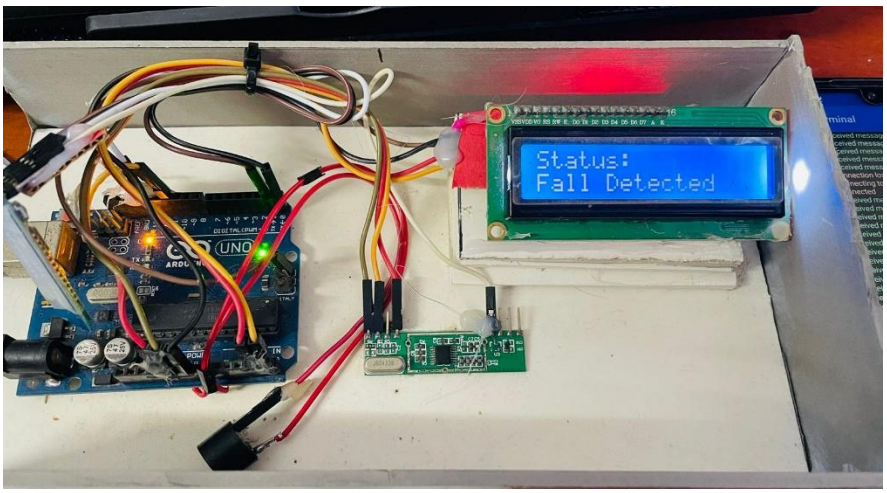


Figure 5.3(b): Fall Detection Output with Buzzer Alert

Figure 5.3: Emergency and Fall Detection Outputs with Buzzer Alerts

Additionally, Bluetooth-based communication was tested using the Serial Bluetooth Terminal application. The transmitted commands were received in real time on a smartphone, enabling remote monitoring of patient needs, as shown in Figure 5.4.

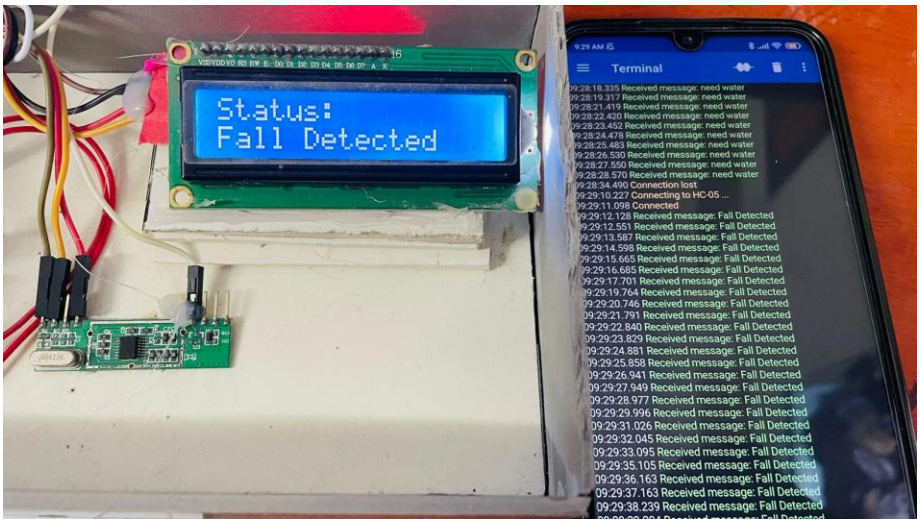


Figure 5.4: Bluetooth Terminal Receiving Real-Time Patient Commands

The implementation successfully demonstrates a functional, wireless assistive system, with outcomes validating accurate gesture recognition, reliable command transmission, and effective emergency alerts, confirming the system's practicality and usability

Chapter 6

Result and Discussion

6.1 Results

The following table 6.1 summarizes the outcomes of the system based on various hand movements detected by the MPU6050 accelerometer, flex sensor inputs, and their corresponding actions:

Table 6.1: Final Output Results of the Project

Condition	Displayed on LCD	Buzzer	Action
Hand is in equilibrium position	No Need	Off	No action taken
Hand tilts/moves to the right	Feeling Sick	Off	Caregiver alerted
Hand tilts/moves to the left	I need water	Off	Caregiver alerted
Hand tilts/moves forward	Washroom	Off	Caregiver alerted
Hand tilts/moves backward	I need food	Off	Caregiver alerted
Finger flex detected (Flex Sensor)	Emergency	On	Immediate caregiver alert
Fall detected (Acceleration Threshold)	Fall Detected	On	Critical caregiver alert

6.2 Threshold Value Calculation

For the MPU6050 Accelerometer Sensor:

Threshold values for tilt detection were determined through calibration and testing to differentiate between deliberate and unintended movements. Below are the conditions:

Tilt to the Left:

```
if (ax > 7000) {
   // Display "I need water"
}
```

Tilt to the Right:

```
if (ax < -4000) {
```

```
    // Display "Feeling Sick"
}
```

Tilt Forward:

```
if (ay> 8000) {
    // Display "Washroom"
}
```

Tilt Backward:

```
if (ay < -8000) {
    // Display "I need food"
}
```

For Flex Sensor:

The flex sensor triggers an emergency alert when the resistance changes due to significant bending:

```
if (flexValue > 600) {
    // Display "Emergency" and activate buzzer
}
```

For Fall Detection:

Acceleration magnitude is calculated to detect falls, using the formula:

magnitude= $\sqrt{ax^2 + ay^2 + az^2}$

Threshold

```
  if ((dax > 4500 || day > 4500 || daz > 4500) && magnitude > 1.2) {
    // Display "Fall Detected" and activate buzzer
}
```

These thresholds ensure that only deliberate gestures or critical conditions like falls trigger the appropriate commands.

6.2 Discussion

The developed system combined gesture-based communication and emergency detection to support paralysis patients in expressing their needs. The MPU6050 accelerometer was used to detect hand tilt directions, and it consistently provided accurate recognition during controlled trials Commands based on tilt were transmitted wirelessly through RF modules and displayed on an I2C LCD without noticeable delay. For emergency detection, a custom-built flex sensor was integrated. Initially, a commercial sensor was considered, but after one of the available flex sensors burned out during testing, an alternative was developed using foil paper and graphite. This improvised sensor performed reliably, triggering emergency alerts without producing false alarms, and demonstrated that cost-effective components can be engineered to replace expensive commercial alternatives.

In addition to gesture-based commands, the system also featured fall detection and remote monitoring. Sudden changes in motion were interpreted as falls, which immediately activated a buzzer to alert caregivers. Normal communication commands, however, were silently logged and displayed, making them accessible on both the LCD and via Bluetooth through the Serial Bluetooth Terminal app. The buzzer was specifically reserved for emergencies to ensure urgency in critical cases. By combining these elements, the system provided both real-time local communication and basic remote accessibility, making it adaptable to home or hospital use within a limited range.

Although the system performed well overall, certain limitations and challenges were observed during testing. One notable issue was the occurrence of false positives in fall detection, especially when rapid hand movements were performed. This could potentially cause confusion in real-world scenarios, and further calibration or algorithmic filtering would be required to minimize such errors. Additionally, the Bluetooth-based communication restricted the caregiver's range of mobility, making the system more suitable for localized environments where the caregiver is nearby. The reliance on the Serial Bluetooth Terminal app introduced occasional delays, meaning caregivers needed to remain alert and check the logs frequently to ensure no patient commands were overlooked. Furthermore, the buzzer only provided alerts for emergencies, requiring

caregivers to depend on the display or app for regular needs, which may reduce efficiency if immediate attention is not given.

Another limitation lies in patient adaptability, as the system requires training for users to learn which specific hand gestures correspond to different commands. While the gestures are simple, this learning curve could initially be challenging for certain patients. Moreover, since testing was performed on healthy individuals under controlled conditions rather than actual paralysis patients, the system's true performance in clinical or home-care scenarios remains to be validated. Despite these constraints, the project demonstrates strong potential as a scalable, cost-effective solution for improving patient–caregiver communication. With further refinements, such as improved fall detection algorithms, extended wireless range, and adaptive gesture recognition, the system can evolve into a more robust and user-friendly assistive technology

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project successfully demonstrated the design and implementation of a low-cost, wearable assistive system for paralysis patients, enabling them to communicate essential needs and emergency conditions through simple hand gestures and finger flexion. The integration of the MPU6050 accelerometer, custom-built flex sensor, RF communication, LCD display, Bluetooth module, and buzzer provided a reliable and practical solution for enhancing patient–caregiver interaction. Importantly, the improvised flex sensor not only reduced the overall cost but also proved to be accurate and dependable during testing, showcasing the potential for resourceful engineering in assistive technologies.

7.2 Future Works

Despite its effectiveness, the system has certain limitations that highlight opportunities for future enhancement. First, the use of Bluetooth restricts communication range; integrating IoT-based connectivity could extend monitoring capabilities for remote caregiving. Second, while the system achieved accurate recognition of hand gestures, false alarms during rapid movements suggest a need for smarter context-aware algorithms to improve reliability. Third, the improvised flex sensor should be replaced with medical-grade sensors to ensure durability, safety, and compliance in clinical settings. Additionally, expanding the gesture set would allow patients to express a wider variety of commands and needs, thereby improving usability. Caregiver notifications could also be strengthened by introducing differentiated audio alerts for each command and developing a persistent command-logging interface, ensuring that no patient request goes unnoticed even when caregivers are momentarily unalert.

In conclusion, this project lays a strong foundation for affordable, practical assistive technologies that can significantly improve the quality of life for paralysis patients. With further refinement in connectivity, accuracy, sensor reliability, and user interaction, the system holds promise for broader

deployment in both home and clinical environments. The findings underscore the importance of combining cost-effectiveness with innovation, offering a pathway toward more inclusive and impactful healthcare solutions.

References:

[1] "Analysis of Muscular Paralysis using EMG Signal with Wavelet Decomposition Approach," ResearchGate. Accessed: Aug. 20, 2025. [Online]. Available: https://www.researchgate.net/publication/364792716_Analysis_of_Muscular_Paralysis_using_EMG_Signal_with_Wavelet_Decomposition_Approach

[2] M. Vijay Kumar, K. Pandurangan, and R. Vinu, "Automated Paralysis Patient Monitoring System," in *2021 IEEE National Biomedical Engineering Conference (NBEC)*, Kuala Lumpur, Malaysia: IEEE, Nov. 2021, pp. 71–76. doi: 10.1109/NBEC53282.2021.9618721.

[3] L. K. Simone, E. Elovic, U. Kalambur, and D. Kamper, "A low cost method to measure finger flexion in individuals with reduced hand and finger range of motion," in *The 26th Annual International Conference of the IEEE Engineering in Medicine and Biology Society*, San Francisco, CA, USA: IEEE, 2004, pp. 4791–4794. doi: 10.1109/IEMBS.2004.1404326.

[4] SpinalCord.com, "Types of Paralysis: Monoplegia, Hemiplegia, Paraplegia, and Quadriplegia." Accessed: Aug. 20, 2025. [Online]. Available: https://www.spinalcord.com/types-of-paralysis

[5] N. Lokhande, S. Dhamale, and C. Railkar, "Flex sensors in healthcare communication: Accessibility and interaction," in *Rehabilitation Engineering, Assistive Technologies and the Future of Healthcare Using Robotics*, Elsevier, 2025, pp. 81–102. doi: 10.1016/B978-0-443-31624-1.00005-8.

[6] G. Morone, A. Pirrera, A. Iannone, and D. Giansanti, "Development and Use of Assistive Technologies in Spinal Cord Injury: A Narrative Review of Reviews on the Evolution, Opportunities, and Bottlenecks of Their Integration in the Health Domain," *Healthcare*, vol. 11, no. 11, p. 1646, Jun. 2023, doi: 10.3390/healthcare11111646.

[7] W. Shi, "Recent Advances of Sensors for Assistive Technologies," *J. Comput. Commun.*, vol. 03, no. 05, pp. 80–87, 2015, doi: 10.4236/jcc.2015.35010.

Appendix:

Transmitter code:

```
#include <SPI.h>
#include <RH_ASK.h>  // RadioHead ASK module for RF communication
#include "I2Cdev.h"
#include "MPU6050.h"
// Create an instance of the MPU6050 sensor
MPU6050 mpu;
// Create an instance of the RH_ASK object for RF communication
RH_ASK driver;
int16_t ax, ay, az, gx, gy, gz;
int flex;
bool blinkState;
// Previous values for detecting spikes
float prevMagnitude = 1.0;
int16_t prev_ax = 0, prev_ay = 0, prev_az = 0;
// For fall latch
unsigned long fallTime = 0;
bool fallActive = false;
void setup() {
  // Initialize Serial communication for debugging
  Serial.begin(9600);
  Serial.println("Initializing MPU...");
  pinMode(A0, INPUT);
  delay(1000);
  // Initialize RF Transmitter (RadioHead)
  if (!driver.init()) {
    Serial.println("RadioHead initialization failed!");
    while (true); // Stop if RadioHead initialization fails
  }
  Serial.println("RadioHead init successful");
  driver.send((uint8_t *)"RH init", strlen("RH init"));
  delay(2000);
  // Start I2C communication
  #if I2CDEV_IMPLEMENTATION == I2CDEV_ARDUINO_WIRE
    Wire.begin();
  #elif I2CDEV_IMPLEMENTATION == I2CDEV_BUILTIN_FASTWIRE
    Fastwire::setup(400, true);
  #endif
```

```
  delay(1000);
  // Initialize MPU6050 sensor
  mpu.initialize();
  Serial.println("MPU6050 conn");
  driver.send((uint8_t *)"MPU conn", strlen("MPU conn"));
  delay(2000);
  if (!mpu.testConnection()) {
    Serial.println("MPU failed!");
    driver.send((uint8_t *)"MPU failed!", strlen("MPU failed!"));
    while (true); // Stay here if MPU6050 connection fails
  } else {
    Serial.println("MPU6050 connection successful");
    driver.send((uint8_t *)"MPU OK", strlen("MPU OK"));
  }
  delay(2000);
  pinMode(LED_BUILTIN, OUTPUT);  // Set up LED for activity indication
}
void loop() {
  // Read the accelerometer and gyroscope data
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
  flex = analogRead(A0);
  // Calculate acceleration magnitude
  float magnitude = sqrt(sq(ax / 16384.0) + sq(ay / 16384.0) + sq(az / 16384.0));
  // Raw axis differences
  int16_t dax = abs(ax - prev_ax);
  int16_t day = abs(ay - prev_ay);
  int16_t daz = abs(az - prev_az);
  // Debugging output
  Serial.print("Flex: "); Serial.println(flex);
  Serial.print("AX: "); Serial.print(ax);
  Serial.print("\tAY: "); Serial.print(ay);
  Serial.print("\tAZ: "); Serial.print(az);
  Serial.print("\tMagnitude: "); Serial.print(magnitude);
  Serial.print("\tΔAX: "); Serial.print(dax);
  Serial.print("\tΔAY: "); Serial.print(day);
  Serial.print("\tΔAZ: "); Serial.println(daz);
  // Default command
  char command[20] = "no need";

  // --- Fall detection (updated thresholds) ---
```

```
  if ((dax > 4500 || day > 4500 || daz > 4500) && magnitude > 1.2) {
    strcpy(command, "Fall Detected");
    fallActive = true;
    fallTime = millis();
  } else if (fallActive && millis() - fallTime < 1000) {
    // Keep fall active for 5 seconds
    strcpy(command, "Fall Detected");
  } else {
    fallActive = false;
    // Other conditions (unchanged)
    if (ax > 7000) {
      strcpy(command, "Need water");
    } else if (ax < -4000) {
      strcpy(command, "Feeling sick");
    } else if (ay > 8000) {
      strcpy(command, "Washroom");
    } else if (ay < -8000) {
      strcpy(command, "Need food");
    } else if (flex > 600) {
      strcpy(command, "Emergency");
    } else {
      strcpy(command, "no need");
    }
  }
  // Send the message over RF
  if (driver.send((uint8_t *)command, strlen(command))) {
    Serial.println(String(command));
  } else {
    Serial.println("Failed to send message");
  }
  // Blink LED to indicate activity
  blinkState = !blinkState;
  digitalWrite(LED_BUILTIN, blinkState);
  // Update previous values
  prevMagnitude = magnitude;
  prev_ax = ax;
  prev_ay = ay;
  prev_az = az;
  delay(500);
}
```

Receiver code:

```
#include <SPI.h>
#include <RH_ASK.h>       // Include the RadioHead ASK module
#include <Wire.h>         // Include the Wire library for I2C communication
#include <LiquidCrystal_I2C.h>  // Include the LiquidCrystal_I2C library
// Create an instance of the RH_ASK object
RH_ASK driver;
// Initialize the LCD with I2C address (default is 0x27 or 0x3F, adjust as needed)
LiquidCrystal_I2C lcd(0x27, 16, 2);  // I2C address, 16 columns, 2 rows
void setup() {
  Serial.begin(9600);
  pinMode(A0, OUTPUT);
  // Initialize the LCD
  lcd.init();          // Initialize the LCD
  lcd.backlight();     // Turn on the LCD backlight
  lcd.clear();
  lcd.print("Waiting...");  // Display a waiting message
  digitalWrite(A0, HIGH);
  delay(1000);
  digitalWrite(A0, LOW);
delay(1000);
  // Initialize the RadioHead driver
  if (!driver.init()) {
    Serial.println("RadioHead initialization failed!");
    lcd.clear();
    lcd.print("RF Init Failed");
    delay(1000);
    while (1);  // Stop if initialization fails
  }
  Serial.println("RadioHead receiver initialized");
  lcd.clear();
  lcd.print("RF Ready...");
  delay(1000);
}

void loop() {
  uint8_t buf[15];  // Buffer to store the received message
  uint8_t len = sizeof(buf);
  // Check if a message is available
```

```
  if (driver.recv(buf, &len)) {
    buf[len] = '\0';  // Null-terminate the string
    String message = String((char*)buf);  // Convert buffer to a String for processing
    // Print the message on Serial Monitor
    Serial.print("Received message: ");
    Serial.println(message);
    // Display the message on the LCD
    lcd.clear();            // Clear the LCD screen
    lcd.setCursor(0, 0);    // Set the cursor to the first line
    lcd.print("Status:"); // Display a label
    lcd.setCursor(0, 1);    // Move the cursor to the second line
    lcd.print(message);     // Display the message
    if(message == "Emergency" || message == "Fall Detected") {
       digitalWrite(A0, HIGH); // Activate external alert (e.g., buzzer)
       delay(1000);
       digitalWrite(A0, LOW);
    }
  }
}
```